Internal Project Report: Trie-Based Word Search and Autocomplete Engine

> **Course:** Data Structures & Algorithms III (DSA 3)
> **Project Title:** Project 15 — Trie-Based Dictionary and Autocomplete
> **Deliverable Source File:** `project/Trie.java`
> **Language & Environment:** Java (Standard Library)

1. Problem Statement

Modern computing environments (search engines, mobile keyboards, text editors) depend heavily on instantaneous prefix lookups and auto-suggestions.

Limitations of Traditional Data Structures:

- **Arrays / Lists:** Finding all words starting with a prefix requires a linear scan across all entries in `O(N × L)` time, where *N* is word count and *L* is word length.
- **Hash Tables (HashMap / HashSet):** While providing `O(L)` exact lookups, hash tables **cannot perform prefix lookups or autocomplete**. To find words matching a prefix like `"th"`, a hash table must scan all *N* stored keys in memory (`O(N × L)`).
- **Redundant Memory:** Storing words such as `"the"`, `"there"`, and `"their"` separately duplicates common prefix characters in memory.

The Solution: Trie (Prefix Tree)

A **Trie** is a tree data structure where each node represents a character. Words sharing common prefixes share the same ancestor nodes. Prefix queries take `O(P)` time (where *P* is prefix length), and autocomplete suggestions run in `O(P + K)` time (where *K* is the number of characters in matching subtree branches), completely independent of total words *N*.

2. Project Objectives

- **Trie Construction:** Build a 26-ary Trie structure for lowercase English letters (`'a'` to `'z'`).
- **Word Insertion:** Store words in `O(L)` time complexity.
- **Exact Word Search:** Verify full word existence in `O(L)` time, differentiating complete words from prefixes.
- **Prefix Search (`startsWith`):** Confirm whether any word begins with a specified prefix in `O(P)` time.
- **Autocomplete Suggestions:** Return all matching words for a prefix using Depth-First Search (DFS) subtree traversal.
- **Empirical Validation:** Execute comprehensive test cases covering exact matches, prefix checks, autocomplete expansion, and edge cases.

3. Data Structure Design & Architecture

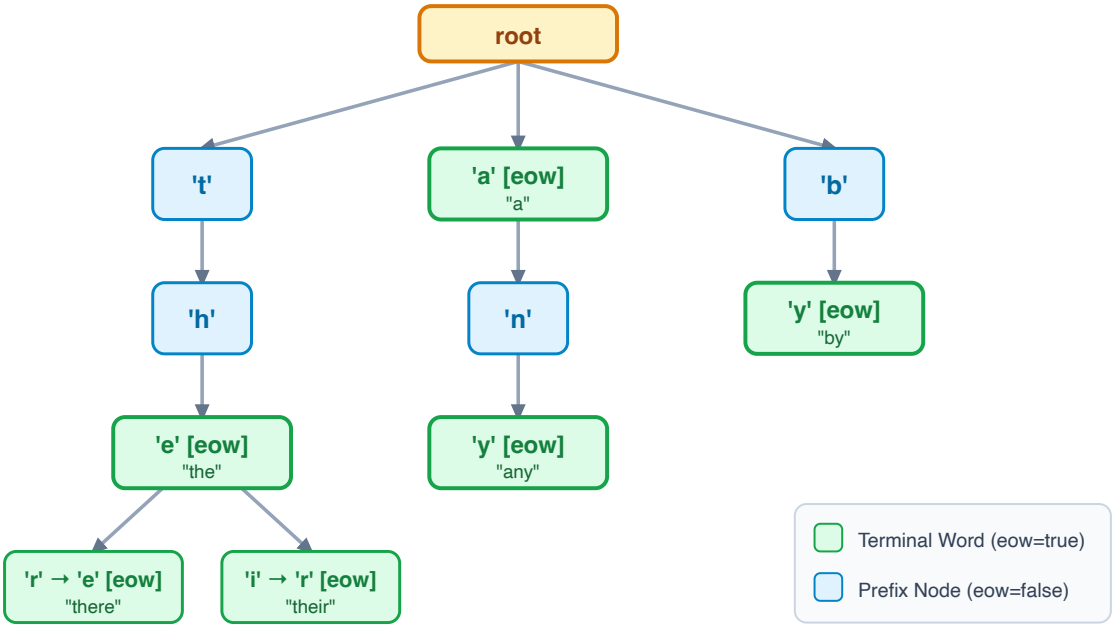
Each node in the Trie represents a character along a prefix path:

```
static class Node {
    Node[] children;
    boolean eow;

    public Node() {
        children = new Node[26]; // Indices 0 to 25 map 'a' through 'z'
        for (int i = 0; i < 26; i++) {
            children[i] = null;
        }
        eow = false;
    }
}
```

- `children`: Array of size 26 referencing child nodes mapped via `idx = ch - 'a'`.
- `eow` (End of Word): Boolean flag set to `true` at the terminal character of a valid word.

Hierarchical Trie Architecture for Vocabulary: {"the", "a", "there", "their", "any", "by"}



4. Algorithm Implementation & Methods

A. Word Insertion (`insert`)

Traverses characters of the word one by one starting from `root`. If a child node at `idx = ch - 'a'` is null, a new node is allocated. At the final character (`i == word.length() - 1`), the flag `curr.children[idx].eow` is set to `true`.

B. Exact Search (`search`)

Follows the character path down the Trie. If any character link is null, the word does not exist (returns `false`). At the last character, returns `true` only if `node.eow == true`, correctly rejecting intermediate prefixes that are not standalone words.

C. Prefix Search (`startsWith`)

Similar to search, but does not check the `eow` flag. If all characters of the prefix exist sequentially in the Trie, it immediately returns `true`.

D. Autocomplete Suggestions (`autocomplete` & `collectWords`)

1. Steps down the Trie to the terminal node of the given prefix.
2. If the prefix exists, invokes recursive Depth-First Search (DFS) helper `collectWords(curr, prefix, results)`.
3. Traverses all non-null children (0 to 25), reconstructing words character-by-character and appending any node where `eow == true` to the suggestions list.

5. Time and Space Complexity Analysis

Operation	Time Complexity	Auxiliary Space	Technical Explanation
Insert	`O(L)`	`O(L)`	Performs *L* node transitions; allocates at most *L* new nodes for unique paths.
Search	`O(L)`	`O(1)`	Follows *L* array pointers without allocating dynamic memory.
Prefix Search (startsWith)	`O(P)`	`O(1)`	Traverses *P* prefix nodes directly.
Autocomplete	`O(P + K)`	`O(K)`	Traverses *P* prefix steps, then visits *K* matching subtree characters via DFS recursion stack.

**Notation: L = length of word, P = length of prefix, K = total characters across matching suggestions.*

6. Empirical Test Suite & Results

The program includes automated test execution in `main()` over dataset: `{"the", "a", "there", "their", "any", "by"}`.

Test Cases:

1. **Exact Search:**
   - `search("their")` → `true` (Validated full word presence).
   - `search("any")` → `true` (Validated full word presence).
   - `search("thi")` → `false` (Correctly detected non-terminal path).
   - `search("abc")` → `false` (Correctly detected missing word).
2. **Prefix Check (startsWith):**
   - `startsWith("any")` → `true`
   - `startsWith("th")` → `true` (Matches "the", "there", "their").
   - `startsWith("abc")` → `false`
3. **Autocomplete Expansion:**
   - `autocomplete("th")` → `[the, their, there]`
   - `autocomplete("a")` → `[a, any]`
   - `autocomplete("xyz")` → `[]` (Edge case: missing prefix handled gracefully).

7. How to Compile and Run

```
# 1. Compile the implementation
javac project/Trie.java

# 2. Run the test suite and demonstration
java project.Trie
```

Actual Execution Console Output:

```
true
true
false
false
true
true
false
Autocomplete for 'th':  [the, their, there]
Autocomplete for 'a':   [a, any]
Autocomplete for 'xyz': []
```